

crewAI: 给 AI 组一支施工队

多智能体协作框架 —— 让一群各有分工的 AI 帮你干活

上游仓库: github.com/crewAIInc/crewAI | 实测版本: crewai 1.15.23 (commit ca1d55abb) | 实测模型:
GLM-4.6 (Anthropic 兼容中转) | 2026-10-01

一句话总览: crewAI 是一个开源 Python 框架, 让你用「**给每个 AI 写一张岗位说明书 (Agent)** → **给每个岗位派一张工单 (Task)** → **把工单串成一条流水线 (Crew)**」三步, 搭出一支能自动协作完成复杂任务的 AI 团队。

1 crewAI 到底是个什么东西？

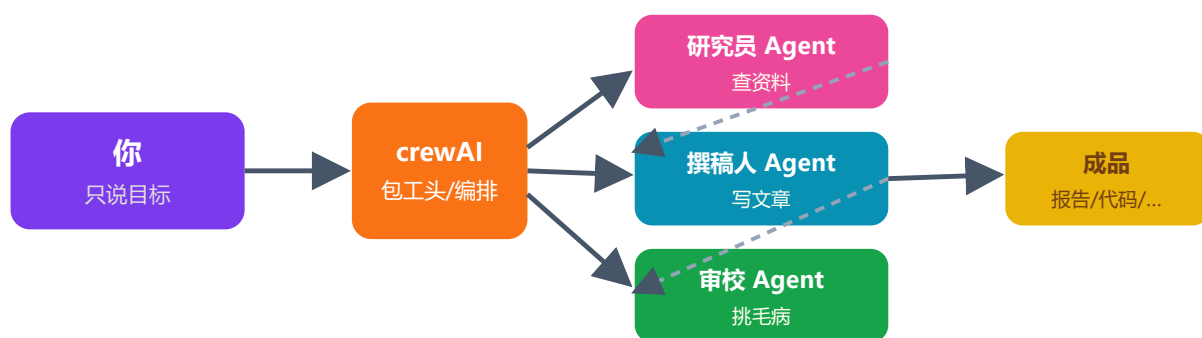
"多智能体编排框架"这七个字每个都认识，连起来就不知道在说什么了。

crewAI = 一套 Python 类库，专门用来"雇一群 AI 员工并管理他们干活"。

拆掉术语墙

智能体 (Agent)：一个能自己思考、自己决定"下一步干什么"、还能调用工具（搜索、查库、读文件）的 AI 程序。和你平时聊天窗里的 AI 的区别是：聊天窗 AI 只会**回答**，Agent 会**动手**。

编排 (Orchestration)：就是"排班 + 分活 + 传话"的管理工作。crewAI 干的就是这个管理工作。



你只说目标，crewAI 负责排班、分活、传话、收货

2 为什么非要"多个" AI? 一个不够吗?

大模型那么聪明, 把所有要求都塞进一条超长提示词不行吗?

一个 AI 干所有活 = 一个人开饭店: 既是厨师又是收银又是采购, 忙中必出错。

没有 crewAI 之前, 人们怎么做?

① **手写提示词链**: 自己写代码把 A 模型的输出粘给 B 模型, 中间全靠手工粘合, 一处改动全部重调; ② **单 Agent 硬扛**: 角色越塞越多, 模型开始"精神分裂"——上一段还在当研究员, 下一段就忘了自己在干嘛。问题的根源: **上下文窗口 (模型一次能"看到"的内容上限) 有限, 塞的角色越多, 每个角色扮演得越糊**。多智能体的解法: **把一个万能但容易犯糊的 AI, 拆成几个窄而专的 AI**。每个 Agent 的提示词里只有自己那一个角色, 注意力集中, 出错率下降; 还能互相校验——这也是实验里那位"研究员"自己总结的第 2 个点。



分工的本质: 让每个模型的上下文里只有"一个角色"

3 Agent 的 role / goal / backstory 是怎么回事？

就是给三个字符串变量赋值，凭什么这就算“招了个员工”？

这三个字段最终会被拼进发给大模型的提示词——所谓“招员工”，其实是给模型发一张详细岗位说明书。

拆开来：**role**（岗位名）让模型进入角色语境；**goal**（目标）告诉它“干成什么样算好”；**backstory**（背景故事）给它注入专业口吻和判断习惯。三段文字会被 crewAI 拼成系统提示词发给模型，模型于是持续用这个身份思考和说话。

实验证据（步骤 3 实测）：同一个任务，把研究员的 role 从“资深技术研究员”改成“武侠小说家”，产出立刻从规范要点变成满纸江湖气——**角色设定就是最强的风格开关**，改一个字符串，改的是模型的整个人格。



Agent 三件套 = 拼一张岗位说明书发给模型

4 Task 是什么？任务的结果怎么传给下一个人？

员工干完活，成果是怎么“交接”的？

Task = 一张工单：description 说做什么，expected_output 定义验收标准，context 指定“开工前先读哪些工单的成果”。

`expected_output` 是 crewAI 的一个亮点：它不是装饰，而是真的会被拼进提示词当**验收标准**——模型会努力把自己的输出修成那个格式。`context=[t1]` 的意思是：t2 开工前，把 t1 的 **Final Answer** 塞进 t2 的上下文，等于把上道工序的半成品递给下道工序。这和你知识库里的 [[构建流水线与Pass]] 是同一个思想：**每级只加工上一级的中间产物。**



工单链：上一张工单的 Final Answer 就是下一张的原料

5 Crew 和 Process：谁来决定干活顺序？

Crew 把 agent 和 task 打包之后，到底按什么顺序执行？

Process 只有两种：sequential（按工单列表顺序传，像流水线）和 hierarchical（先派一位"经理 Agent"统筹分派）。

sequential：完全确定， $t1 \rightarrow t2 \rightarrow t3$ 依次执行——这就是你熟悉的流水线模式，顺序是代码写死的。

hierarchical：crewAI 会额外造一个"经理 Agent"，它读所有任务后**自己决定**派给谁、检查谁的结果，甚至打回重做（必须配 `manager_llm`）。代价是 token 翻几倍；收益是面对"事先无法排死工序"的活更灵活。



左边是流水线，右边是项目部

6 Agent 执行任务时，底层到底发生了什么？

verbose=True 时刷屏的那些 Thought / Action / Observation 是什么？

Agent 的执行引擎是一个循环：Thought（想）→ Action（动）→ Observation（看结果）→ 再想……直到给出 Final Answer。这个套路叫 ReAct。

crewAI 把"任务描述 + 岗位说明书 + 可用工具清单"打包发给模型；模型回复的可能是"我要调用某工具"（Action），框架就去真的执行工具，把结果（Observation）追加进对话再发回去；如此循环，直到模型认为完工，输出 Final Answer。你看到的"AI 在自主工作"，本质就是这个由框架托管的思考-行动循环。



ReAct 循环：所有"自主智能"都来自这三拍的反复播放

7 工具 (Tool) : 只会写字的模型怎么"动手"?

模型明明只会生成文字, 怎么查数据库、发请求?

秘密是: 模型只负责"说我要调哪个函数、参数是什么", 真正动手的是框架; 干完把结果用文字塞回去。

流程: ① 你把 Python 类 (继承 `BaseTool`, 实现 `_run` 方法) 挂到 Agent 上; ② 框架把工具的名字和描述翻译成文字写进提示词; ③ 模型在循环里说"调用 `canteen_menu_query`"; ④ 框架执行真正的 Python 函数; ⑤ 结果以文字回填进上下文。模型从头到尾**一个字节的数据库都没碰过**——它只是个会点菜的顾客, 厨房是框架。

实验证据 (步骤 4 实测): "食堂推荐官" Agent 拿到一件菜单查询工具后, 日志出现 `Tool Output: 红烧肉 12元; 番茄炒蛋 8元...`, 随后推荐"番茄炒蛋 8 元 + 米饭"——**价格只可能来自工具返回** (模型自己根本不知道"本食堂"的菜价), 这就是工具调用生效的铁证。



模型点菜 (说调用名), 框架炒菜 (跑函数), 结果回锅 (进上下文)

8

它怎么接上我自己的模型？litellm 是干嘛的？

各家 API 格式都不一样，crewAI 每家都要写适配器吗？

不。crewAI 把“翻译各国语言”的活外包给 litellm——一个统一网关库，模型名统一写成 provider/model 格式即可。

`LLM(model="anthropic/glm-4.6", base_url=..., api_key=...)` 的意思是：把请求按 Anthropic 协议格式发往指定中转地址，跑模型叫 glm-4.6。OpenAI、Gemini、Ollama 本地模型.....都是换个名字的事。

⚠ **实测踩坑（值得记一辈子）**：crewAI 1.x 起 **litellm 不再默认安装**！裸装 `pip install crewai` 后跑非原生模型直接 `ImportError`。正确姿势：`pip install "crewai[litellm]"`。另外中文 Windows 控制台默认 GBK，crewAI 日志带 emoji 会炸，跑之前 `PYTHONUTF8=1`（实验 1 首跑日志全是乱码就是它）。

9 Flow 又是什么？和 Crew 什么关系？

README 一会儿说 Crew 一会儿说 Flow，它们是竞争关系吗？

不是竞争，是两种管理哲学：Crew = 自治小组（模型自己商量怎么干）；Flow = 确定性事件流水线（代码写死谁在什么条件下被触发）。

Flow 用 `@start`（起点）`@listen`（监听某方法完成就触发）`@router`（条件路由）装饰普通 Python 方法，方法之间通过共享状态（`self.state`，Pydantic 模型）传数据。**Flow 里的步骤可以是纯 Python（零 token）也可以内嵌一个 Crew。**这就是你知识库里 [[确定性脚本]] 思想的 Agent 版：**能确定的不要求 AI，必须 AI 的才交给 AI。**生产系统的标准姿势：Flow 做骨架（控制流、状态、分支），Crew 做肌肉（真正需要动脑的环节）。



`@listen(receive_topic)` 触发 `@listen(write_draft)`——事件驱动，代码说了算

Flow = 骨架（确定），Crew = 肌肉（智能），合体才是生产级

实测：step5_flow.py 全程零 LLM 调用跑通，事件链 `@start` → `@listen` → `@listen` 依次触发，日志逐个打印 `Flow Method Completed`。

10 Memory 和 Knowledge 是什么？

AI 员工下班就失忆吗？第二天还记得昨天干了什么吗？

Crew 可挂两层记忆：Memory 管"经历"（之前跑过什么、学到了什么），Knowledge 管"资料"（你塞给它的文档，用 RAG 随查随用）。

Memory（记忆）：短期记忆让一次运行内的 Agent 共享中间结论；长期记忆把历史运行的经验存下来，下次类似任务直接复用。**Knowledge**（知识）：把 PDF、txt 等文件交给 Crew，框架切块建索引，Agent 干活时像查字典一样检索相关片段——这叫 RAG（检索增强生成）。没有它俩，Agent 是金鱼；有了，才像老员工。

Memory 记忆 = 经历

短期：本次运行内的共享便签
长期：跨次运行的经验存档

Knowledge 知识 = 资料

你的 PDF/网页 → 切块建索引
Agent 干活时检索（RAG）

记忆是"我的经历"，知识是"我的图书馆"

11 crewAI 和 LangGraph 怎么选？

听说都是做 Agent 编排的，为什么要存在两个？

一句话：crewAI 把"组队"做成默认（快）；LangGraph 把"流程图"做成默认（控）。

	crewAI	LangGraph
心智模型	招一支角色化团队（公司）	画一张状态机流程图（图）
控制粒度	Crew 自治为主，Flow 补控制	每个节点每条边都你说了算
上手速度	极快，十几行跑通	较慢，要先学图模型
适合	角色分工明确的协作型任务	分支/回环/人工审批多的严格流程

有趣的是你仓库里正躺着 langgraph 项目——两者都学后可以互相印证：**crewAI 的 Flow 本质上也是一张隐式状态图**，只是用装饰器代替了显式画图。

这个项目给我什么

① 十几行代码搭 AI 工作组

把"研究→写作→审校"这类多角色任务自动化，不用手写任何提示词编排代码。

② 给 Agent 装手脚

继承 BaseTool 写个 Python 函数，Agent 就能查库、调 API、读文件。

③ 换模型像换电池

GLM、GPT、Claude、本地 Ollama，改一个字符串全队换装。

④ 产出格式可验收

expected_output / output_pydantic 让下游拿到结构化数据而非一坨文本。

⑤ 复杂系统有骨架

Flow 状态机 + Crew 内嵌，写出能上生产的确定性 workflow。

⑥ 结合你已学的流水线思想

sequential = [[构建流水线与Pass]] 的 AI 版，可以直接拿它重做 img2threejs 式任务拆解。

主实验：亲手组建你的第一支 AI 工作组

实验目标：从零装环境 → 跑通双 Agent 流水线 → 亲眼看工具调用 → 对比 Flow。全部命令在 **Windows 11 + Python 3.12** 实测通过，直接复制即可。

步骤 0 · 建环境（一次即可）

```
py -3.12 -m venv .venv # crewAI 要求 Python >=3.10,<3.14, 3.14 不行!
.venv\Scripts\python -m pip install "crewai[litellm]" # 注意带 [litellm], 否则接第三方模型报 Import
.venv\Scripts\python -c "import crewai; print(crewai.__version__)"
```

怎么操作 在 `crewai\exercise\first_crew\` 目录依次执行上面三条。

目的 隔离出 Python 3.12 环境，装上框架 + litellm 适配层。

实现 得到 .venv 虚拟环境和 crewai 1.15.23 包。

验证 最后一条命令打印版本号即成功。

✅ 预期结果：终端输出 **crewai 1.15.23**（实测如此）。若报 `ImportError: litellm not ...`，说明漏了 [litellm] 后缀。

步骤 1 • 跑通双 Agent 流水线 (step2_crew.py)

```
PYTHONUTF8=1 .venv\Scripts\python step2_crew.py # Git Bash; PowerShell 用 $env:PYTHONUTF8=1
```

脚本定义**研究员**和**撰稿人**两个 Agent、两张 Task（撰稿人 `context=[t1]` 接研究员的产出），Process.sequential 顺序执行，模型走 `anthropic/glm-4.6` + 中转 `base_url`。

怎么操作 直接运行脚本（需先设好 ANTHROPIC_BASE_URL / ANTHROPIC_AUTH_TOKEN 环境变量）。

目的 亲眼看到两个角色接力完成一件事。

实现 日志依次出现两位 Agent 的思考过程，最后打印成品。

验证 结尾出现“最终产出：”和一段完整的介绍文字。

✅ 预期结果：最终产出为一段 200 字以内的大白话介绍，如实测原文：“想象一下小组作业：有人定计划，有人动手做，还有人负责检查。多智能体协作就是把这套办法交给 AI——让几个各有分工的 AI 组成‘团队’……几个 AI 互相校验，就像同学互查作业，能少犯错，把事情办得更靠谱。”

步骤 2 • 改一个角色，看人格切换 (step3_role_change.py)

```
PYTHONUTF8=1 .venv\Scripts\python step3_role_change.py
```

和步骤 1 唯一区别：研究员的 role/goal/backstory 换成了**武侠小说家**（要求每点配一个武侠比喻）。

怎么操作 运行脚本；再用编辑器打开对比 step2_crew.py 里 researcher 的三行设定。

目的 验证“角色设定 = 风格开关”。

实现 同样的任务，产出完全换了一种人格口吻。

验证 研究员的 3 个要点里出现武侠比喻。

✅ 预期结果（实测原文）：研究员要点全是武侠比喻，撰稿人成品——“就像武侠片里高手结阵：每个成员各管一摊，有的负责收集信息，有的负责分析判断，有的负责动手执行……三个臭皮匠顶个诸葛亮，分工加配合，再复杂的事也能拆开来干成。”

步骤 3 • 给 Agent 发工具，看它"动手" (step4_tool.py)

```
PYTHONUTF8=1 .venv\Scripts\python step4_tool.py
```

"食堂推荐官" Agent 挂了一件自定义 `BaseTool` (不联网的假菜单数据库)。脚本里 `class MenuTool(BaseTool)` 的 `_run` 方法就是工具本体。

怎么操作 运行脚本，盯着日志找 "Tool Output" 字样。
目的 理解 ReAct 循环里 Action/Observation 两拍。
实现 Agent 先查菜单再推理推荐，价格来自工具而非模型记忆。
验证 日志出现菜单内容和基于菜价的推荐。

✅ 预期结果 (实测原文) : Tool Output: 红烧肉 12元; 番茄炒蛋 8元; 清炒时蔬 6元; 糖醋里脊 13元 → 最终产出: "推荐: 番茄炒蛋 (8 元) + 剩余 2 元加个米饭....."

步骤 4 • Flow: 一条不花 token 的流水线 (step5_flow.py)

```
PYTHONUTF8=1 .venv\Scripts\python step5_flow.py
```

三个方法用 `@start` / `@listen` 串成事件链，中间一步是纯 Python (零 LLM 调用)，体会"Crew 自治、Flow 确定性"的差异。

怎么操作 运行脚本，对照日志数一数发生了几次 LLM 调用 (答案: 0 次)。
目的 感受事件驱动的确定性执行。
实现 `@start` 触发 `@listen` 触发 `@listen`，共享 state 层层传递。
验证 打印 `[start]/[listen]` 三行日志和最终大纲文本。

✅ 预期结果 (实测原文) : Flow 最终输出: 《crewAI 与 LangGraph 的区别》写作大纲: 1. 定位差异 2. 心智模型 3. 选型建议 (本步为占位，未调用 LLM)，且日志逐个方法打印 ✅ Flow Method Completed。

实测踩坑清单 (都是真金白银踩出来的)

坑	现象 → 解法
Python 3.14 装不上	crewAI 要求 <code>>=3.10,<3.14</code> ，系统默认 3.14 直接出局 → 用 <code>py -3.12 -m venv</code> 建 3.12 环境。
litellm 不默认安装	crewAI 1.x 起改为可选依赖，接 GLM/DeepSeek 等非原生模型报 ImportError → 装 <code>"crewai[litellm]"</code> 。
GBK 控制台乱码/报错	日志含 🚀 emoji，GBK 编不出来刷 Sync handler error，输出成乱码 → 脚本里 <code>sys.stdout.reconfigure(encoding="utf-8")</code> + 环境变量 <code>PYTHONUTF8=1</code> 。
中转站接入	Anthropic 兼容中转不用改代码 → <code>LLM(model="anthropic/模型名", base_url=中转地址, api_key=令牌)</code> 三参数即可。
限流撞车	多实验并行跑 + IDE/会话同时用同一个 key，中转返回 <code>rate_limit_error 1302</code> ，重试耗尽后 Crew 直接报 Task Failed → 实验串行跑、错峰重试即可 ，不是代码 bug。

本指南由 Claude Code 按 learn-project 流程产出 | 源码阅读: repo/lib/crewai/src/crewai/ (crew.py 的 kickoff、process.py、flow/flow.py、llm.py) | 实验日志存于 exercise/first_crew/*.log | 知识入库: vault/项目笔记/crewai.md